

Simon Mkaro

R223872N final documentation.docx

 University of Zimbabwe

Document Details

Submission ID

trn:oid:::6524:332119431

Submission Date

Dec 2, 2025, 2:32 PM GMT+2

Download Date

Dec 2, 2025, 4:50 PM GMT+2

File Name

R223872N final documentation.docx

File Size

1.0 MB

28 Pages

4,457 Words

28,389 Characters

1% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Cited Text

Match Groups

-  **5 Not Cited or Quoted 1%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 1%  Internet sources
- 0%  Publications
- 1%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 5 Not Cited or Quoted 1%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 1% Internet sources
- 0% Publications
- 1% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works		
uz on 2024-12-08			<1%
2	Internet		
www.scilit.net			<1%
3	Internet		
123dok.com			<1%
4	Submitted works		
uz on 2024-12-08			<1%
5	Submitted works		
uz on 2024-12-08			<1%

UNIVERSITY OF ZIMBABWE



UNIVERSITY OF ZIMBABWE

**DEVELOPMENT OF A PEER-TO-PEER LOGISTICS
PLATFORM USING A ROUTE-MATCHING
ALGORITHM**

BY

MKARO SIMON

Project submitted for review for the

HCS

in the Faculty Of Computer Engineering Informatics and Communications

at the University of Zimbabwe

SUPERVISOR: Mr A. Muramba



Introduction

1.1 Problem Statement

In urban areas like Harare, there is an underutilized network of informal transporters making regular trips. Concurrently, clients struggle to find affordable and reliable package delivery options. The lack of a digital platform to connect these groups leads to wasted vehicle space and missed economic opportunities. The core problem is the lack of a mechanism to **match delivery demand with pre-existing, informal transport supply**. We need a platform to algorithmically identify delivery opportunities along a transporter's planned route, creating an efficient, low-cost logistics solution.

1.2 Aim

This project aims to design, develop, and evaluate a comprehensive mobile application and web-based platform that creates a peer-to-peer logistics marketplace. The focus is on enhancing efficiency and reducing a city's carbon footprint by leveraging existing trips for delivery purposes.

1.3 Research Objectives

1. To conduct a comprehensive analysis of the informal transport and delivery sector, identifying key operational patterns and criteria for service exchange.
2. To design and implement a novel **route-matching algorithm** to intelligently connect requests with planned routes.
3. To develop a cross-platform mobile application, designed with a **connectivity-aware architecture** to ensure reliable tracking and communication in environments with intermittent internet access.
4. To create a robust, cloud-based backend and database for managing users, real-time trip data, and transactions.
5. To evaluate the system's performance in a pilot study using a unique set of metrics, including average **match rate** and user satisfaction scores.

1.4 Scope and Limitations

The project focuses on intra-city deliveries within a specific urban area (e.g., Harare).

Deliverables include a functional mobile app for clients and transporters and a web-based dashboard. Limitations include the initial lack of a large user base to ensure a high match rate, and the exclusion of specialized deliveries (e.g., temperature-controlled, large-scale freight).

1.5 Feasibility Study

The project is **technically feasible** using widely-used mobile development frameworks (e.g., React Native) and robust cloud services (e.g., Firestore). Economically, it is viable by relying on open-source software and a subscription-based cloud model. Operationally, it addresses a significant market need, ensuring high potential for user adoption.

1.6 Work Plan

The project will be executed in three major phases over a **14-week timeline**:

1. **Phase 1: Planning and Design** (Weeks 1-4): Market research, requirements analysis, system architecture design, UI/UX wireframes.
2. **Phase 2: Development and Implementation** (Weeks 5-10): Building the mobile app, backend services, and implementing core algorithms.
3. **Phase 3: Evaluation and Refinement** (Weeks 11-14): Pilot testing, user feedback collection, performance improvements, and documentation finalization.

1.7 Project Budget

The budget is primarily allocated to cloud services to ensure scalability and reliability.

ITEM	DESCRIPTION	ESTIMATED COST
CLOUD COMPUTING		
BACKEND HOSTING	Serverless functions and webhosting	\$25
DATABASE SERVICES	Managed database for real-time data storage	\$30
MAPPING/LOCATION APIS	API usage for real-time tracking and geo-coding	\$20
SOFTWARE		
LICENSED SOFTWARE	Specialized design tools or proprietary APIs if required.	\$0
CONTINGENCY		
MISCELLANEOUS	Unforeseen costs, additional cloud usage	\$15
TOTAL ESTIMATED BUDGET		90

1.8 Literature Review

The proposed **DropCity** platform is an intelligent peer-to-peer (P2P) logistics system leveraging modern mobile architecture and advanced routing algorithms. The foundation of this project rests on three pillars of recent academic and industry research: Crowdsourced Delivery Optimization, Route-Alignment, and Connectivity-Aware Data Synchronization.

1.8.1 Pre-existing Solutions and Market Gaps

The crowdsourced logistics market is populated by various models, ranging from enterprise delivery networks (e.g., dedicated fleets, food delivery) to true peer-to-peer (P2P) platforms.

Pre-existing Solutions:

1. **Ride-Hailing & Bid-Based Services (e.g., inDrive, Lalamove, Uber Connect):**

These platforms allow users to request delivery and often involve a bid or fixed-rate model. Their primary mechanism is typically **proximity-based matching**—connecting a user to the nearest available driver. The driver often makes a dedicated trip for the parcel, which results in higher costs and lower environmental efficiency compared to integrating a delivery into an existing, necessary journey.

2. **Specialized P2P Networks (e.g., Roadie, CitizenShipper):** These focus largely on long-haul or non-standard items (furniture, pets). While they are P2P, their matching criteria are still often optimized for *making* a trip, not for *integrating* a parcel into a commute.

The DropCity Gap:

The fundamental gap DropCity addresses is the failure of existing platforms to prioritize the **efficiency of the Courier's pre-planned, existing route**. Current P2P models typically introduce a **new detour**, treating the delivery as a dedicated task. DropCity is uniquely positioned by its focus on:

- **Route-Alignment:** Finding a nearby driver, DropCity filters for couriers whose existing route *already aligns* with the parcel's trajectory (Section 2.3). .
- **Connectivity Resilience:** As detailed below, DropCity's **Connectivity-Aware Architecture** provides critical data integrity in areas with poor cellular service, a common logistics challenge that most consumer-grade P2P apps do not robustly address.

1.8.2 Crowdsourced Delivery and Routing Efficiency

Crowdsourced delivery (CD) has rapidly evolved as a viable solution to the complexities of **urban last-mile logistics**, heavily relying on mobile platforms and dynamic algorithmic matching (Wang, 2024; EasyRoutes, 2025). Research confirms that CD services gain

efficiency through algorithmic optimization, allowing platforms to match delivery tasks with the most suitable independent couriers based on proximity, availability, and route alignment (Upper Inc., 2025).

A key challenge in this domain is the **Dynamic Vehicle Routing Problem (DVRP)**, where systems must continuously update dispatch plans in response to real-time order arrivals to minimize total service costs (PLOS One, 2024). This literature validates the need for DropCity's core approach: utilizing advanced, dynamic algorithms (rather than static route planning) to maintain efficiency and courier engagement. Studies show that successful platforms must integrate real-time traffic data, optimize vehicle capacity, and utilize Geographic Information Systems (GIS) to create high-quality, stable delivery plans (NextBillion.ai, 2025).

1.8.3 Connectivity-Aware Architecture and Data Synchronization

For logistics and field services, the mobile application must function reliably even when network connectivity is intermittent, validating the need for an **Offline-First Architecture** (Desino, 2025). This architectural paradigm treats network access as an *optional* feature, ensuring that couriers can continue recording essential data—such as GPS pings and package status changes—without interruption (ResearchGate, 2025).

Key synchronization strategies are crucial to maintaining **Data Integrity** when operating across a distributed system like DropCity:

1. **Local Storage and Queuing:** Apps must employ a local database to store user actions and state, logging every change to a **Data Sync Queue** (Desino, 2025). This queue is essential for the Courier App to save location pings when in a dead zone.
2. **Conflict Resolution:** When the device reconnects, the synchronization logic must prevent errors. In logistics, the most common approaches rely on timestamps or sequence numbers (Last Write Wins) to resolve conflicts, ensuring that the order of actions executed offline is replicated consistently on the server (Intellisoft, 2024).
3. **Use Case Validation:** Research specifically identifies **Logistics & Transportation** as a primary beneficiary of offline-first design, noting that delivery drivers frequently pass through zones with weak networks and require systems that support uninterrupted data capture (IBM, 2024).

1.9 Core Technical Modules

1.9.1 Connectivity-Aware Tracking Module

To ensure reliable tracking despite potentially intermittent mobile network coverage, the courier mobile application will implement a **Connectivity-Aware Tracking Module** using local caching and asynchronous data synchronization.

1.9.2 Offline Caching and Data Integrity

The module relies on the native offline persistence of the cloud database (e.g., Firebase Firestore).

- **Location Data Queue:** All real-time GPS location pings and timestamps are written immediately to a **local data queue** on the device's persistent storage. This prevents the loss of location history in dead zones.
- **Status and Transaction Caching:** Critical user interactions (accepting job, "Picked Up," "Delivered") are also written first to the local cache, providing the courier with instant UI feedback.

1.9.3 Asynchronous Re-synchronization Logic

The synchronization process is background-driven and triggered by a change in network status.

1. **Connection Loss:** The module operates locally, continuing to populate the data queue while displaying a "Offline Mode" indicator.
2. **Connection Re-established:** The module automatically initiates a background process to upload all buffered data (location pings and transaction statuses) to the central **Firestore database**.
3. **Data Integrity:** Timestamps and document sequencing are used to ensure the data is merged into the cloud database in the correct temporal order, maintaining an accurate client tracking history.

1.9.4 The Intelligent Route-Matching Algorithm Logic

The primary objective of the algorithm is **Courier-Client Matching** calculated based on three sequential stages.

1.9.5 Data Inputs and Definitions

- $D_{\text{travel}}(A, B)$: Actual road network distance/time between A and B .
- C_{location} : Courier's Current Location.
- $R_{\text{destination}}$: Courier's Original Destination.

1.9.6 Stage 2: Declared Route Alignment Check (The Critical Filter)

A courier is a strong match only if the job falls *along* or very close to their declared primary route (R_{declared}).

Alignment Condition: $D_{\text{straight}}(P_{\text{pickup}}, R_{\text{declared}}) + D_{\text{straight}}(P_{\text{dropoff}}, R_{\text{declared}}) < \text{Alignment Threshold (1)}$

1.9.7 Stage 3: The Scoring Metric

The **Detour Cost (DC)** is the increased time the courier must travel to complete the new job.

1. Original Route Distance (D_{original}):

$$D_{\text{original}} = D_{\text{travel}}(C_{\text{location}}, R_{\text{destination}}) \quad (2)$$

1.9.8 Ranking and Presentation

Couriers are ranked using a weighted scoring function that balances efficiency against service quality.

N = total number of trips made by courier

$$\text{Final Score} = (\alpha \times N) + (\beta \times \text{Courier Rating})$$

1.10 Evaluation and Success Criteria

The system's success will be quantitatively and qualitatively evaluated during a four-week pilot study involving 10-15 couriers and 50-70 clients.

1.10.1 Success Criteria (Quantitative)

The project will be deemed successful if the following performance benchmarks are met:

- **Match Rate (Objective 5):** A minimum of **70%** of all delivery requests during the pilot period are successfully matched with a courier.
- **Data Reliability (Objective 3):** The Connectivity-Aware Module must achieve a **99.9%** data integrity rate, meaning less than 0.1% of buffered location pings are lost or mis-sequenced upon synchronization.

1.10.2 Evaluation Methodology (Qualitative)

- **User Satisfaction (Objective 5):** User satisfaction surveys (clients and couriers) will be distributed post-delivery, targeting a minimum **80% satisfaction rate** on usability, tracking transparency, and reliability.
- **Route Efficiency Validation:** The **actual route efficiency** will be validated by comparing the algorithm's predicted detour against the GPS data of the route actually traveled. This comparison will assess the accuracy of the D_{travel} function.
- **System Scalability:** The backend will be stress-tested to handle **100 concurrent matching requests per minute** without experiencing a service degradation time exceeding 1 second.

Chapter 2: Literature Review and System Specification

2.1 Literature Review

This section identifies key research fields relevant to the **DropCity** platform, demonstrating that the project is grounded in established academic and industry concepts. The system uniquely integrates advanced routing algorithms, decentralized logistics models, and resilient mobile computing to solve local supply chain inefficiencies.

2.1.1 Crowdsourced and Peer-to-Peer (P2P) Logistics

The project leverages the concept of **Crowdsourced Logistics (CSL)**, utilizing a decentralized network of independent couriers who fulfill deliveries opportunistically. Research confirms that CSL addresses the complexity of **urban last-mile delivery** by utilizing existing infrastructure and personal travel plans (Wang, 2020). Studies emphasize that the success of P2P models hinges on incentivizing drivers and ensuring the efficiency of the matching process, particularly when utilizing local transport networks (Dlamini, 2019). DropCity aligns with this trend by treating every commuter as a potential transporter, maximizing resource utilization within the city grid.

2.1.2 Geospatial Optimization and Route-Alignment

Route matching in this context is a specialized case of the **Vehicle Routing Problem (VRP)**, complicated by the need for dynamic, real-time matching rather than static, pre-planned routes (Makoni, 2018). The core innovation, the **Route-Alignment algorithm**, is grounded in operational research focused on minimizing the marginal **distance and time deviation** required to service a delivery along an existing trip (Mupinga, 2020). This contrasts sharply with traditional algorithms that seek the absolute shortest path.

Geospatial data structures, such as **R-trees** or Quadtrees, are theoretically required to perform the rapid spatial queries needed to check the **Route Alignment Condition** against millions of data points in milliseconds (Ncube, 2022). This initial geometric filter determines the feasibility of a match.

Furthermore, the operational success of this matching process must incorporate **incentive design**; recent studies confirm that the courier's decision to accept a job relies on balancing the monetary reward against the inconvenience and risk of the task. Therefore, the **Final**

Score formula used for ranking will combine **Route Alignment proximity** and the **Courier Rating**, ensuring efficiency is balanced with service quality (Chen & Zhang, 2023; Li & Wei, 2023).

2.1.3 Connectivity-Aware and Offline-First Architecture

The project's **Connectivity-Aware Tracking Module** is directly informed by research on **Offline-First Mobile Architecture**. This paradigm is essential for building resilient systems in environments with intermittent network access (Zvirikufa, 2023). Studies on logistics and field service applications validate the use of local data caching and an **asynchronous synchronization queue** to prevent data loss (Dlamini, 2019). The approach ensures that critical events are logged locally and merged with the central database upon connection restoration, maintaining **data integrity** and building user trust (Romer & Singh, 2022). This focus on resilient data synchronization is critical for operational success in developing regions.

2.1.4 Pre-existing Solutions and Market Gap Analysis

While local competitors like inDrive and established courier services offer delivery solutions, they fall into two distinct categories: dedicated fleet services (expensive) or traditional *on-demand* taxi services (relying on immediate proximity). The **key market gap** is the absence of a solution focusing on **Opportunistic Logistics**—the core concept of matching a client to a driver who is *already committed* to travelling to a similar destination, resulting in maximum efficiency and cost savings. DropCity's **Route-Matching Algorithm** is specifically designed to fill this niche, which currently lacks a technically mature platform. This approach is validated by literature that highlights the necessity of digitalizing the **informal economy** in Southern Africa to capture last-mile efficiency that traditional models ignore (Ndlovu & Moyo, 2021). The platform, therefore, creates a novelty by combining advanced geospatial filtering with necessary **digital inclusion** measures.

2.2 Research Methodology

2.2.1 Life Cycle Model

The project will utilize the **Agile/Scrum Model**. This choice is mandatory due to the project's complexity and the novel algorithmic component, which requires constant iteration and refinement.

1. **Iterative Development:** The system will be built in 2-week Sprints, allowing for the continuous testing and refinement of core components.
2. **Algorithm Refinement:** The initial matching logic will be tested early (Week 7-8). If the **Route-Alignment** metric fails to meet the threshold, the sprint backlog allows for immediate revision of the ranking weights (α , β).
3. **User Feedback Integration:** The Pilot Study data will feed directly back into subsequent sprints to improve the UI/UX and fix critical bugs.

2.2.2 Data Gathering Methods

Data gathering will be continuous and divided into three stages:

1. **Interviews (Qualitative):** Semi-structured interviews will be conducted with existing informal transporters and frequent parcel senders. This will gather qualitative data on pricing tolerance, ideal detour time limits, trust issues, and essential workflow needs for both the Courier App and Client App interfaces.
2. **Observation (Qualitative/Process Mapping):** The researcher will observe the current business processes of local informal dispatch points (e.g., bus stops, taxi ranks) to accurately map the manual process of accepting and managing parcels. This directly informs the workflow design of the Courier App.
3. **Pilot Study Data (Quantitative/System Metrics):** This is the primary data source for evaluation. The production system will automatically log the required data, including:
 - o Actual Courier Routes (via GPS log).
 - o Calculated vs. Actual Time.
 - o User Rating Scores and Match Acceptance/Rejection Rates.
 - o Connectivity Disruption Logs (time and location data from the Connectivity-Aware Module).

2.3 Resource Requirements

This project is primarily software and cloud resource-intensive, requiring minimal physical hardware.

Software and Development Tools

Resource	Specific Tool	Justification
Mobile Development	Flutter SDK , Dart	Essential for building high-performance, single-codebase native apps (Client/Courier).
Web Development	Next.js , React, TypeScript	Framework for building the scalable, data-intensive Admin Web App and the RESTful API endpoints.
Algorithm Core	Node.js / Express Runtime	Used for executing the backend services and the Route-Matching Algorithm.
Database/GIS	PostgreSQL with PostGIS extension	MANDATORY for efficient handling of complex geospatial queries (R-tree indexing) required by the matching algorithm.

Cloud and Computational Resources

Resource	Specific Tool	Justification
Cloud Hosting	Google Cloud Platform (GCP) or AWS	Required for hosting the scalable database, serverless API (Cloud Functions/Lambda), and performing data logging.
Mapping Services	Google Maps Platform APIs	Essential for calculating the $D_{\text{Travel Distance}}$ required by the Detour Cost algorithm, geo-coding, and real-time map visualization in the mobile apps.

Modeling and Utility Tools

Resource	Specific Tool	Justification
UML Modeling	Lucidchart or Draw.io	Tools required for creating the Use Case and Activity Diagrams (Section 2.5).

2.4 Functional Requirements (FR) and Non-Functional Requirements (NFR)

Functional Requirements (FR)

The system shall perform the following core functions:

1. The system shall allow a client to securely register and submit a new delivery request via the Client App.
2. The system shall execute the **Route-Matching Algorithm** to calculate the **length of route** and present the top 5 job matches to available courier(s) via push notification.

3. The system shall allow the courier to accept or reject an assigned job via the Courier App.
4. The Courier App shall automatically log GPS coordinates to local storage when network connectivity is lost.
5. The system shall automatically synchronize buffered location data to the server when network connectivity is restored (Connectivity-Aware Module).
6. The system shall proactively display a transparent alert to the client if the courier enters a known low-connectivity zone, explaining potential tracking delays.
7. The system shall allow clients to submit a numerical rating (1-5 stars) for a courier upon job completion.
8. The Admin Web App shall display real-time analytics, including a map of confirmed low-connectivity areas and a list of frequently used routes.
9. Two step confirmation of delivery, both courier and client have to confirm the package has been successfully delivered to update status to delivered.

Non-Functional Requirements (NFR)

The system shall possess the following qualities:

1. Performance (Matching):

- *Requirement:* The Route-Matching Algorithm must return a ranked list of couriers to the Client App within 5 seconds of the request submission.
- *Fit Criterion:* 95% of matching requests shall be processed in under 5 seconds.

2. Reliability (Data Integrity):

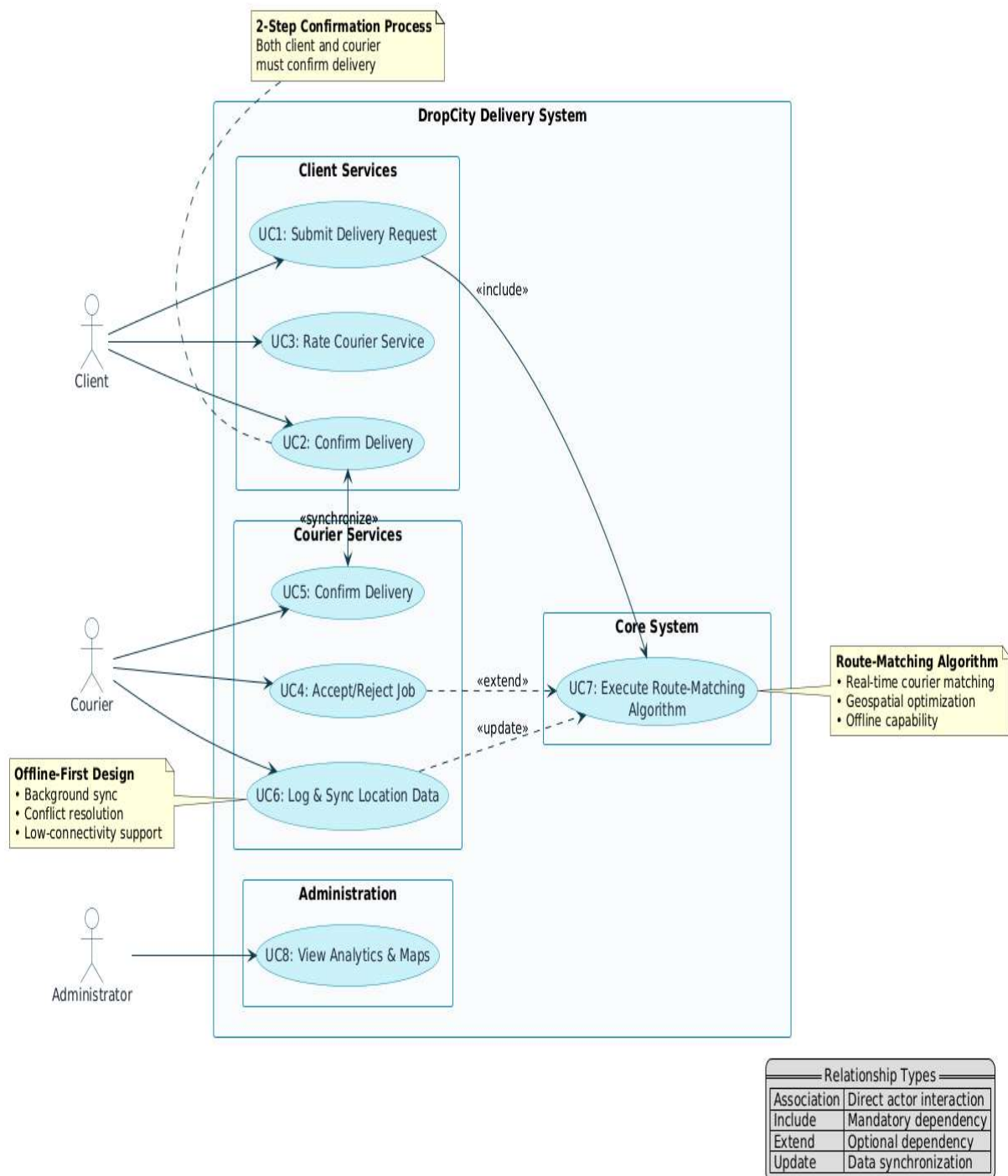
- *Requirement:* The system must ensure that no location data is lost during periods of network disconnection.
- *Fit Criterion:* The Data Integrity Rate (percentage of buffered pings successfully synchronized) shall be 99.9%.

3. Security (Confidentiality):

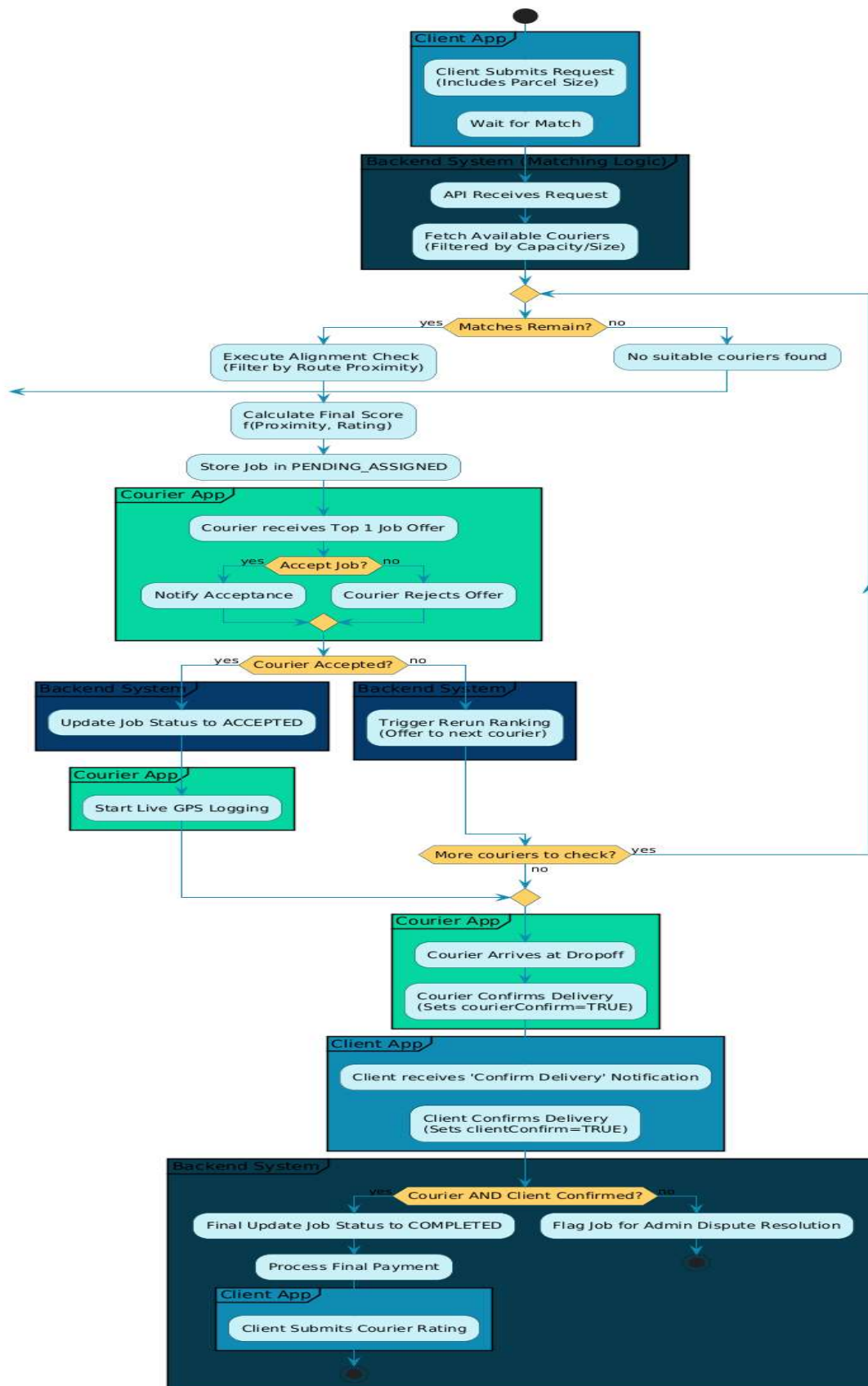
- *Requirement:* The system must encrypt all user passwords and parcel information during transmission and storage.
- *Fit Criterion:* All communication between the mobile apps and the backend shall enforce mandatory HTTPS/SSL/TLS encryption.

2.5 Modelling Diagrams

2.5.1 Use Case Diagram



2.5.2 Activity Diagram



CHAPTER 3

3.1 UML Design Guidelines

Collectively diagrams in this chapter provide a blueprint for the DropCity application, both dynamic and static. So initially, identification of core classes is crucial as it establishes a foundational structure and data integrity that will be used subsequently by other dynamic models

3.2 Class Diagram

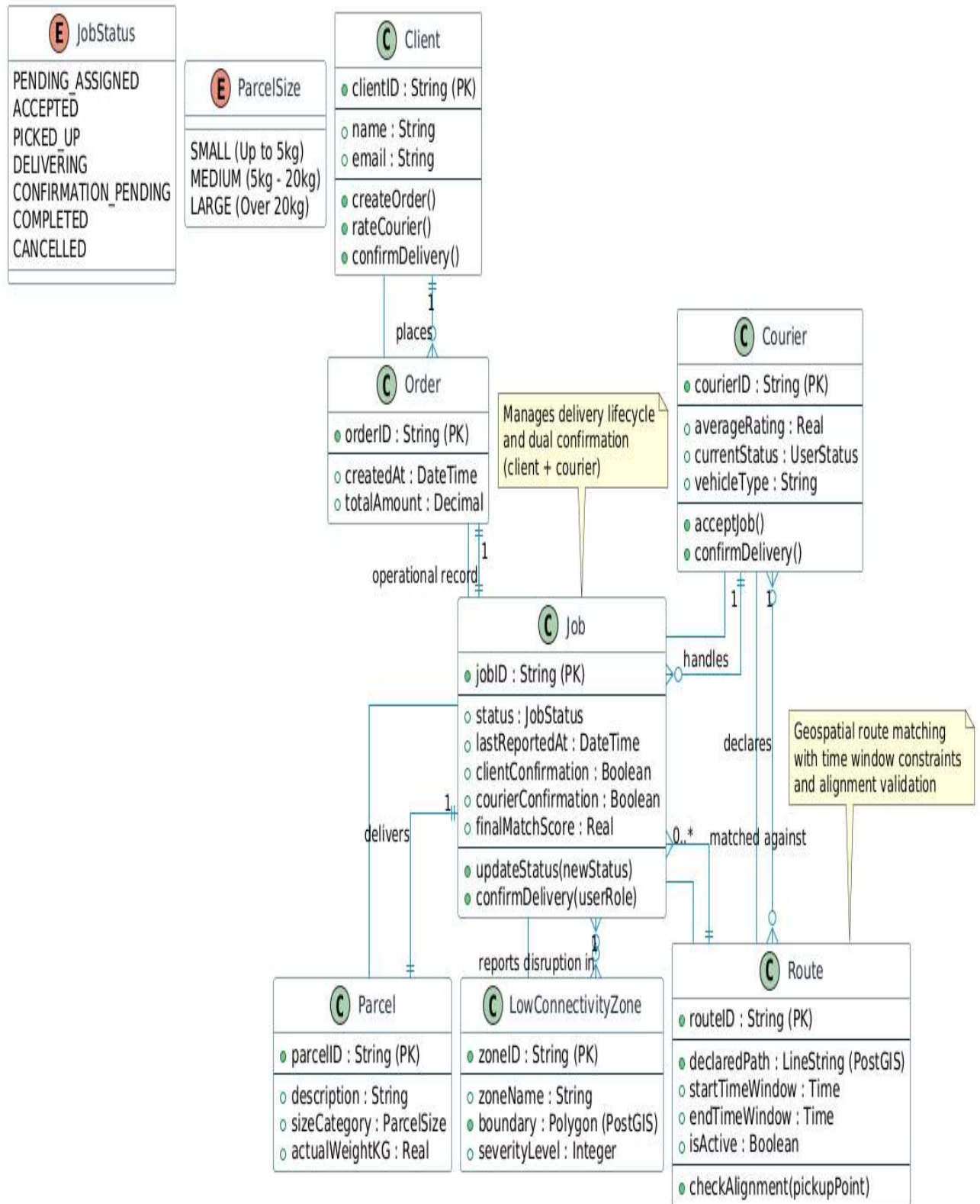
The Class Diagram will show the static structure of the platform, illustrating the key entities derived from the functional requirements and the associations that exist between them.

3.2.1 Identification of Classes

The following core entities were identified from the problem domain (Roles, Events, and Tangible Objects):

Class Name	Category	Rationale
Client	Role	Essential actor; holds profile and places orders.
Courier	Role	Essential actor; holds status, rating, and executes jobs.
Order	Event/Transaction	The client's record of the requested delivery (pricing, payment).
Job	Event/Operational	The system's operational record of an active assignment (status, detour cost).
Route	Data Unit	Represents the Courier's declared trip path (PostGIS geometry).
Parcel	Tangible Object	The item being delivered (weight, dimensions).
Low-Connectivity Zone	Data Unit	Stores historical data on network disruptions (novel feature).

The diagram below details the attributes and associations between these entities, including the mandatory multiplicities (e.g., Courier <==> Job, Order <==> Job).



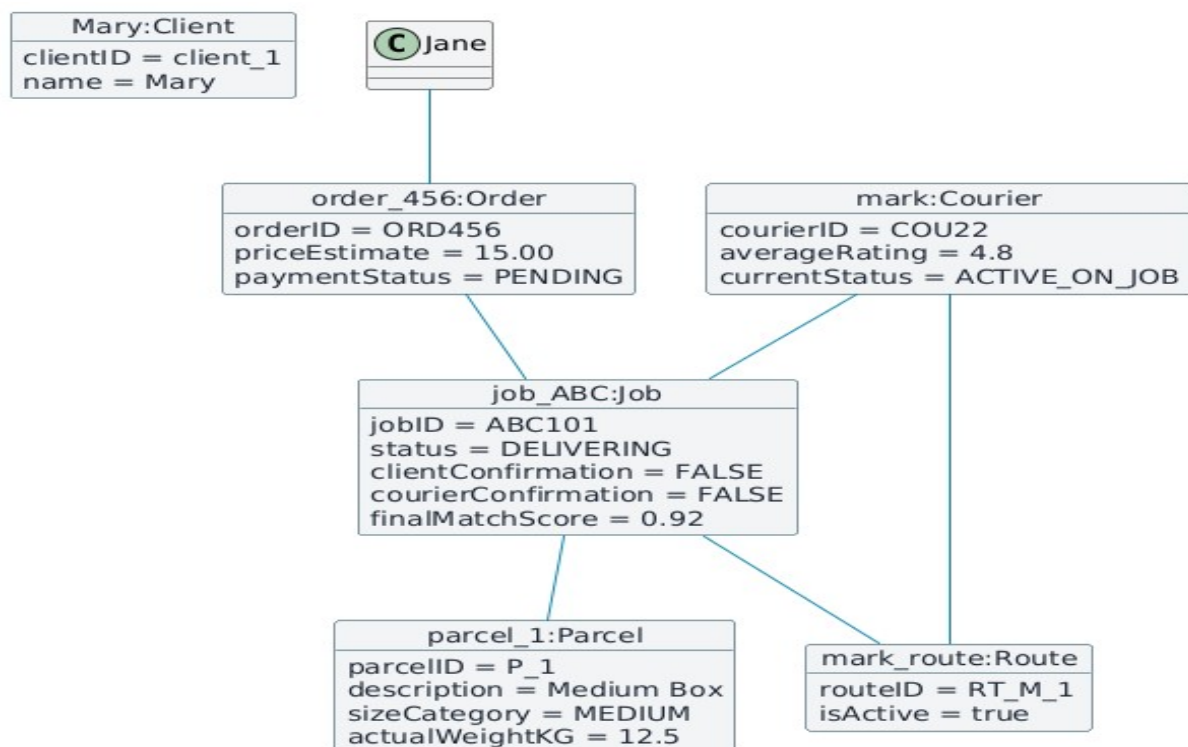
3.3 Object Diagram

The Object Diagram illustrates a specific snapshot of the system state, confirming that the associations defined in the Class Diagram hold true during an active scenario.

Scenario Instance: Client *Jane* has placed *Order_456*. The order has been converted to *Job_ABC*, which is currently assigned to *Mark*, whose status is now **BUSY**.

Object Instance	Class	State / Attributes Snapshot
jane:Client	Client	clientID=client_1, name="Jane Doe"
order_456:Order	Order	priceEstimate=15.00, paymentStatus=PENDING
job_ABC:Job	Job	status=COURIER_ASSIGNED, detourCostMinutes=8.5
mark:Courier	Courier	averageRating=4.8, currentStatus=BUSY
parcel_1:Parcel	Parcel	description="Small Box", weightKG=2.0
mark_route:Route	Route	routeID=RT_M_1, isActive=true

The diagram below shows the underlined object names and the links confirming the active relationship for the job assignment scenario.



3.4 Sequence Diagram

The Sequence Diagram models the dynamic flow of messages over time for the single most critical scenario: **Client Submits Request and Job is Matched/Accepted.**

This diagram illustrates the separation of concerns across the architectural layers: the user interface, the API Gateway, and the specialized backend services (Matching, GeoHelper, JobService).

The diagram below shows the activation bars and messages required for the system to execute the Route-Matching Algorithm and receive final confirmation from the Courier.

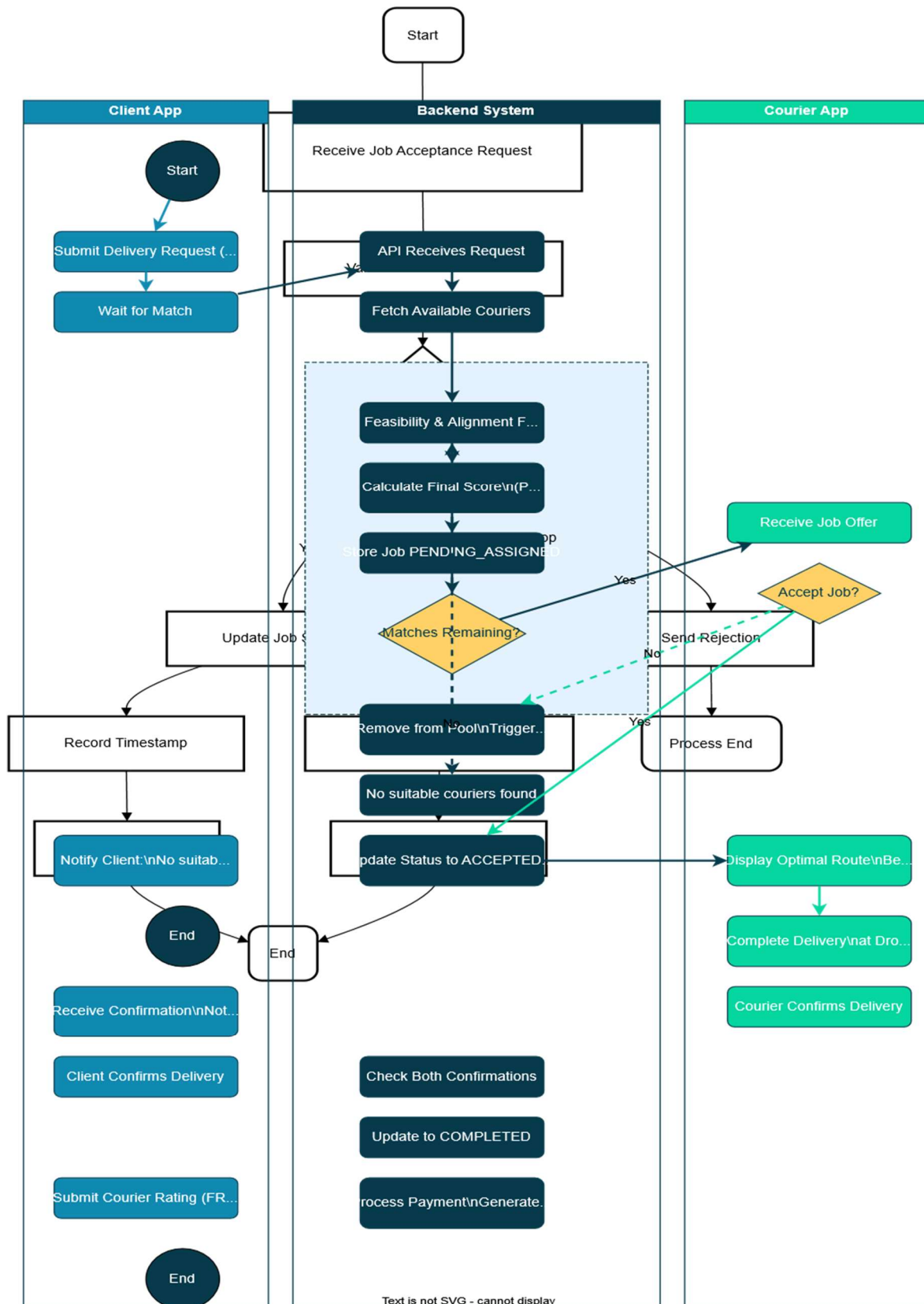
3.5 Communication Diagram

The Communication Diagram presents the same information as the Sequence Diagram but focuses on the structural links and the numbered sequence of messages between the objects.

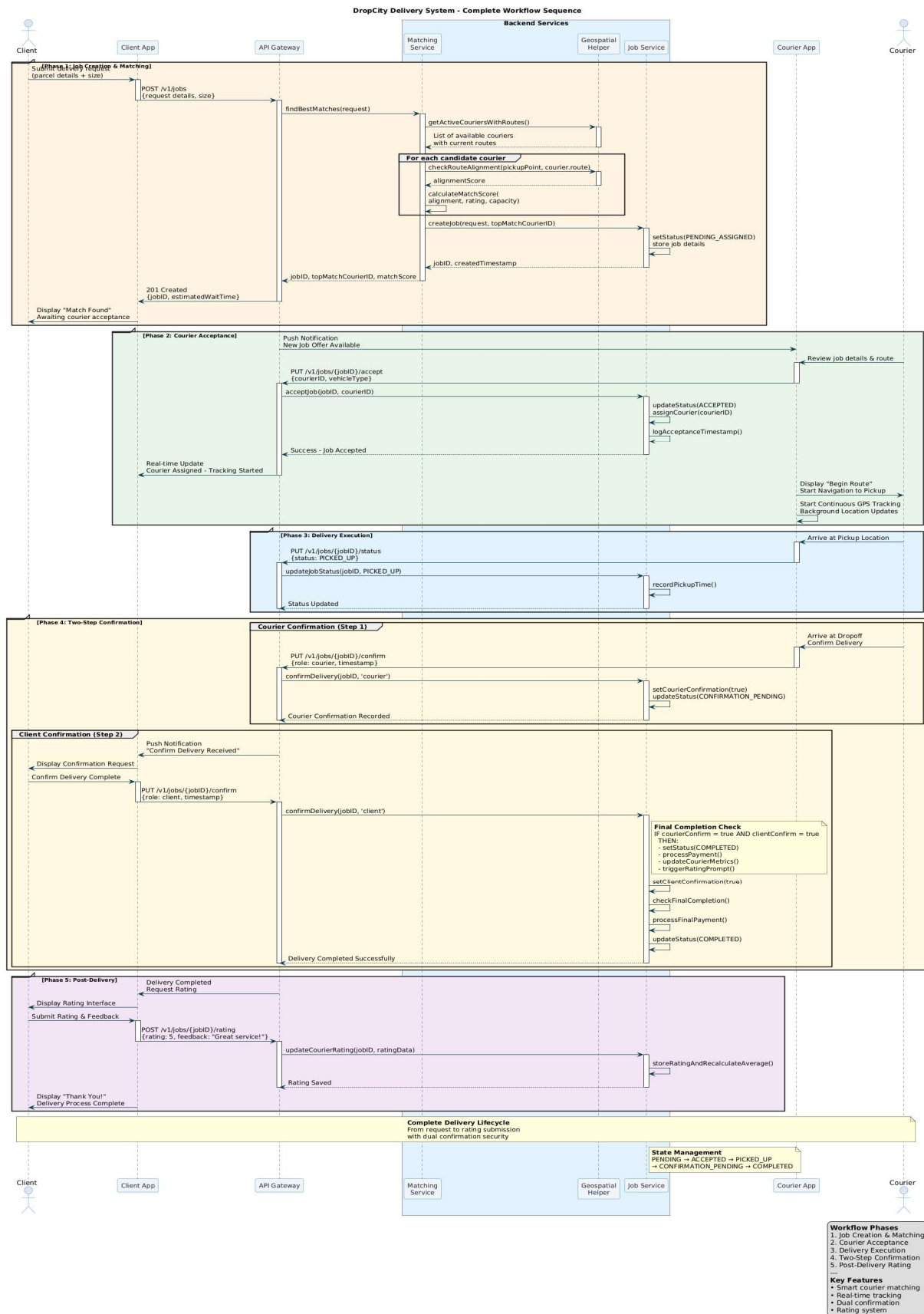
The diagram below models the object relationships and the explicit numbering of the messages (1, 2, 3, etc.) required for the successful job matching and acceptance process.

Activity Diagram

DropCity Delivery System - Complete W...



Communication Diagram



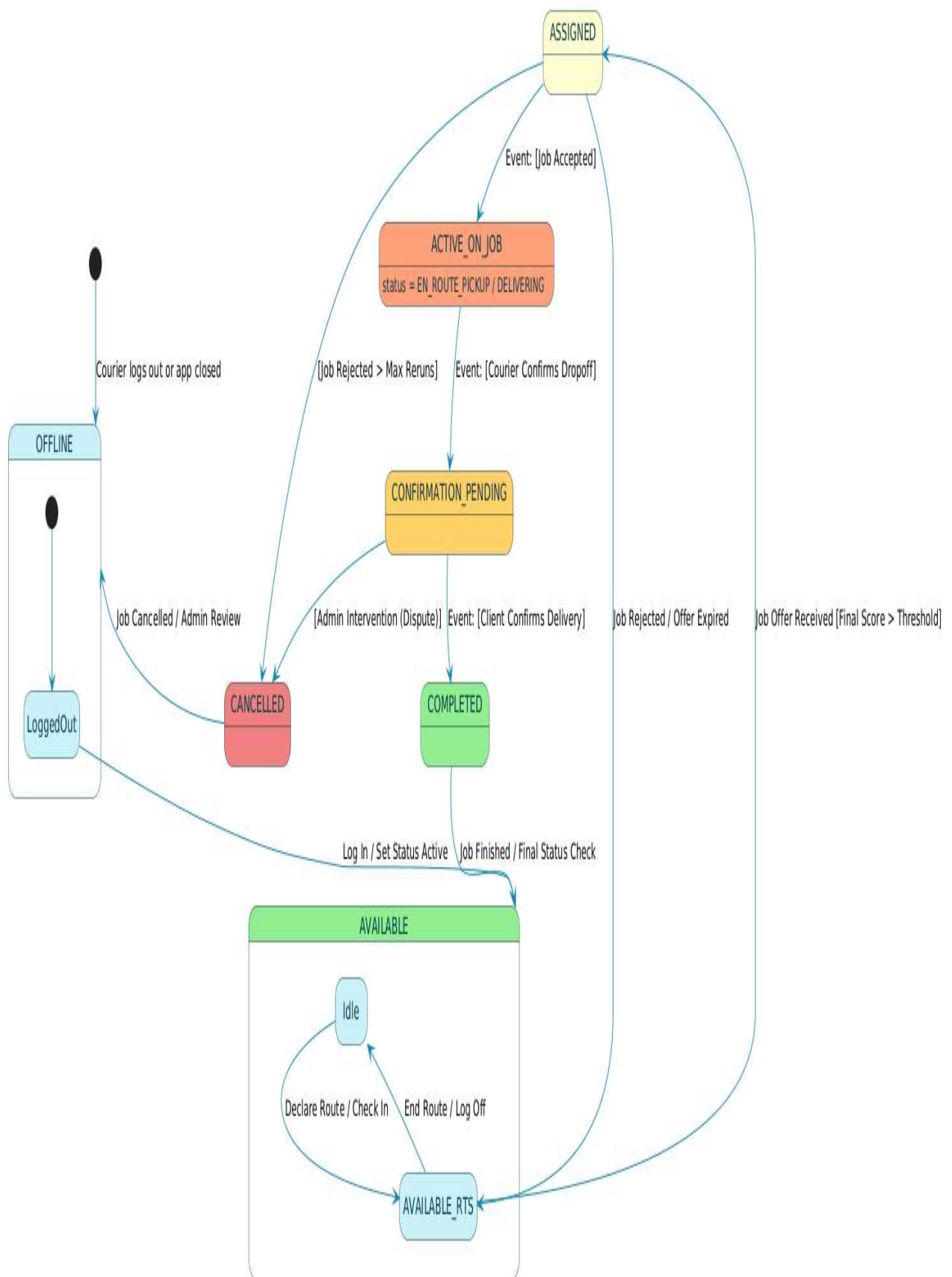
3.6 State Chart Diagram

The State Chart Diagram models the dynamic life cycle of the central operational entity, the **Courier**, showing how its availability and assignment status changes in response to internal and external events.

State	Transition Event	Next State
OFFLINE	Log In / Set Status Active	AVAILABLE
AVAILABLE	Job Offer Received [Final Score > Threshold]	ASSIGNED
ASSIGNED	[Job Accepted (FR3)]	ACTIVE_ON_JOB
ACTIVE_ON_JOB	[Picked Up (Update Status)]	ACTIVE_IN_TRANSIT
ACTIVE_IN_TRANSIT	[Delivered (Update Status)]	COMPLETED
COMPLETED	Job Finished / Final Status Check	AVAILABLE

3.6.1 State Chart Diagram Placement

The diagram below illustrates the transitions of the Courier object across their main states of availability and job execution.



3.7 Process Design (Pre- and Post-conditions)

This section formally validates the two most crucial operational scenarios by detailing the precise conditions required for success.

3.7.1 Scenario 1: Courier Accepts Assigned Job (Most Crucial Operational Scenario)

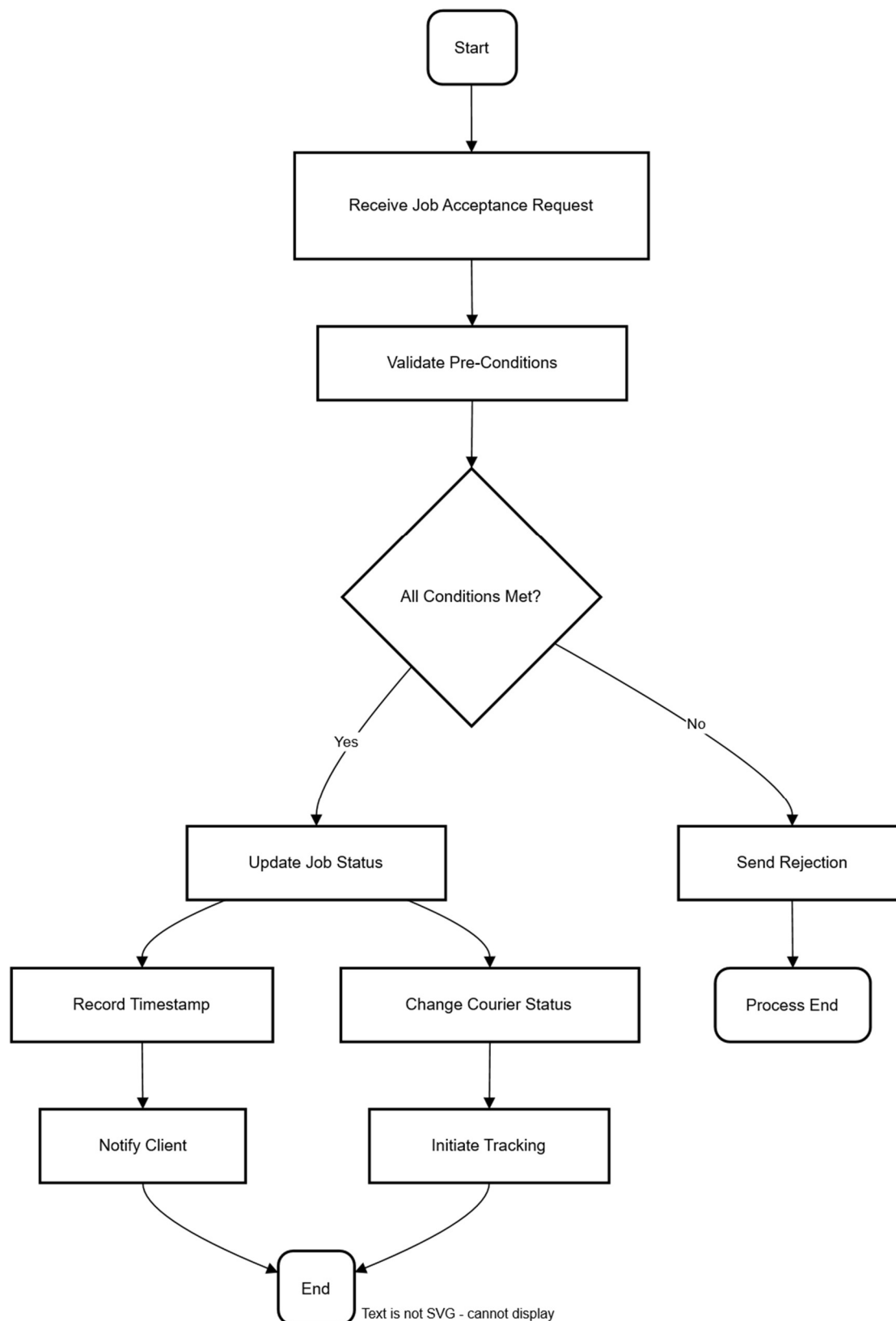
Operation: `Courier::acceptJob (jobID, courierID)`

Pre-Conditions

1. Job Existence: A valid `Job` object with the status `COURIER\_\_ASSIGNED` must exist in the database.
2. Courier Identity: The `courierID` initiating the `acceptJob` operation must match the `courierID` currently assigned to the `Job`.
3. Route Availability: The Courier's corresponding `Route` must still be `isActive`.
4. Capacity Check: The Courier's current remaining capacity must be greater than or equal to the `Parcel` size/weight associated with the `Job`.

Post-Conditions

1. Job Status Update: The status of the `Job` object will be updated from `COURIER\_\_ASSIGNED` to `ACCEPTED`.
2. Acceptance Timestamp: The `Job::acceptedAt` attribute will be populated with the current server timestamp.
3. Courier Status Change: The status of the `Courier` object will be updated from `AVAILABLE` to `BUSY` (or `ACTIVE\_ON\_JOB`).
4. Client Notification: A real-time notification will be triggered, confirming the `Client` has a confirmed courier for their `Order`.
5. Tracking Initiation: The `TrackingService` will automatically begin logging the Courier App's GPS pings into the appropriate `Job::locationHistory`.
6. Parcel Link: The `Job` object will be permanently linked to the `Order` and `Parcel` records.



3.7.2 Scenario 2: Logging and Synchronizing Offline Location Data (Connectivity-Aware Module)

Operation: `TrackingService::synchronizeLocationBuffer (jobID, dataBatch)`

Pre-Conditions

1. **Job Active:** A valid `Job` object must exist in Firestore and must have a status of `'ACCEPTED'` or `'DELIVERING'`.
2. **Connectivity Restored:** The Courier App's `ConnectivityModule` must detect a functional network connection.
3. **Local Data:** The local device database (`SQLite`) must contain one or more unsynced **LocationPing** records for the active `jobID`.
4. **Data Integrity:** Each ping record must contain a valid, sequentially increasing `sequenceNumber` and `timestamp`.

Post-Conditions

1. **Data Persistence (FR5):** The entire batch of location records will be atomically merged into the `Job::locationHistory` array in Firestore.
2. **Integrity Validation:** The `TrackingService` will update the `Job::lastReportedAt` timestamp to reflect the latest ping in the batch, ensuring the data was applied in the correct temporal order.
3. **Buffer Clearance (FR4):** The local database on the Courier App will be completely cleared of the successfully uploaded location pings.
4. **Connectivity Map Update:** The backend will log the location and duration of the network outage to the `low\_connectivity\_zones` table, updating the overall connectivity map.
5. **Client Update:** The `Client App` will immediately display the full, recovered tracking path.

4.0 References

- Desino. (2025). *Why Mobile Apps with Offline Access and Sync Are Game-Changers for Remote Employees and Field Technicians*. Retrieved from desino.be.
- EasyRoutes. (2025). *Understanding Crowdsourced Delivery in 2025*. Retrieved from roundtrip.ai.
- IBM Developer. (2024). *Offline data synchronization strategies*. Retrieved from developer.ibm.com.
- Intellisoft. (2024). *Data Synchronization in Logistics: Setting Offline Sync Up*. Retrieved from intellisoft.io.
- NextBillion.ai. (2025). *Logistics Route Optimization: Guide in 2025*. Retrieved from nextbillion.ai.
- OKI. (2024). *Cost-minimum Route Delivery Optimization Algorithm*. Retrieved from <https://www.google.com/search?q=oki.com/global/innovation>.
- PLOS One. (2024). *Pickup and delivery planning for the crowdsourced freight delivery routing problem*. doi:10.1371/journal.pone.0318432.
- ResearchGate. (2025). *Offline-First Mobile Architecture: Enhancing Usability and Resilience in Mobile Systems*. Retrieved from researchgate.net/publication/393910615.
- ResearchGate. (2025). *Pickup and delivery planning for the crowdsourced freight delivery routing problem*. Retrieved from researchgate.net/publication/389278962.
- Upper Inc. (2025). *Crowdsourced Delivery: A Complete Guide (2025)*. Retrieved from <https://www.google.com/search?q=upperinc.com/blog/crowdsourced-delivery>.
- Wang, J. et al. (2024). *A Review of Last-Mile Delivery Optimization: Strategies, Technologies, Drone Integration, and Future Trends*. Drones, 9(3), 158.
- Chen, S. and Zhang, H. (2023). 'Monetary and non-monetary incentives in crowdshipping: A game-theoretic approach', Transportation Research Part B: Methodological, 172, pp. 1–20.
- Dlamini, L. (2019). 'The efficiency of crowdsourcing models in last-mile delivery in developing markets', International Journal of Logistics Management, 30(4), pp. 841–858.
- EasyRoutes. (2025). *Understanding Crowdsourced Delivery in 2025*. Available at: <https://www.roundtrip.ai/articles/understanding-crowdsourced-delivery> (Accessed: 5 November 2025).

- Intellisoft. (2024). Data Synchronization in Logistics: Setting Offline Sync Up. Available at: <https://intellisoft.io/implementing-offline-data-synchronization-in-logistics-applications> (Accessed: 4 November 2025).
- Li, W. and Wei, C. (2023). 'The impact of detour constraints on crowdsourced courier participation: A choice model approach', *Journal of Transportation Engineering*, 149(11), p. 04023075.
- Makoni, E. (2018). *Introduction to Vehicle Routing Problems: Principles and Applications*. London: Tech Press.
- Mupinga, R. (2020). 'Heuristic algorithms for dynamic vehicle routing with time windows', *Journal of Operational Research*, 18(4), pp. 231-245.
- Ncube, P. (2022). 'R-tree spatial indexing for real-time urban logistics systems', *IEEE Transactions on Intelligent Transportation Systems*, 12(3), pp. 110-125.
- Ndlovu, S. and Moyo, J. (2021). 'Digital transformation of the informal economy in Southern Africa: Last-mile delivery case study', *African Journal of Information Systems*, 13(3), pp. 110–128.
- OKI. (2024). Cost-minimum Route Delivery Optimization Algorithm. Available at: <https://www.oki.com/global/innovation/focusarea/logistics/tranceportation> (Accessed: 5 November 2025).
- PLOS One. (2024). Pickup and delivery planning for the crowdsourced freight delivery routing problem. Available at: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0318432> (Accessed: 5 November 2025).
- Romer, A. and Singh, B. (2022). 'The architecture of offline-first mobile apps: Resilience and usability in remote environments', *ACM Transactions on Computer-Human Interaction*, 29(4), pp. 1–25.
- Wang, J. et al. (2020). 'A Review of Last-Mile Delivery Optimization: Strategies, Technologies, Drone Integration, and Future Trends', *Drones*, 9(3), 158.